MDPI

Article

PBEA: Policy-Bound Composition of State-Transition Integrity and Runtime Confinement

SeongSu Park [1] and Ki-Hyung Kim [2]

[1] Department of Computer Engineering, Ajou University, Suwon, Republic of Korea; parky@ajou.ac.kr
[2] Department of Cyber Security, Ajou University, Suwon, Republic of Korea; kkim86@ajou.ac.kr
* Corresponding author designation requires author confirmation before submission.

Abstract

An authorized request can violate business invariants or exercise undeclared resources. Policy-Bound Execution Architecture (PBEA) composes candidate-state enforcement, runtime confinement, component policy identities, and explicit failure evidence. A Rust/OpenBSD implementation distinguishes application completion from supervisor-observed native termination and policy identity from actual kernel configuration. Re-analysis reproduces an archived 1,080-record four-mode corpus, which demonstrates complementary state and runtime boundaries. A separate 540-record comparison against unbound composition retains both layers while varying component identities: bound composition rejects all 120 declared mismatch trials, whereas unbound composition accepts them. Both configurations permit a deliberately mislabeled installation, establishing the limit of identity checks. Boundary tests show that allowed external transmissions can survive candidate Abort and that preopened descriptors retain authority. A new five-mode benchmark retains 150,000 raw operation timings in 300 independent process blocks across three workloads. It reports absolute and relative costs without treating within-process samples as independent experiments. The contribution is a reproducible integration contract under trusted-code and kernel assumptions, not universal attack prevention, kernel attestation, or atomic rollback of external I/O.

Keywords: policy enforcement; state integrity; runtime confinement; OpenBSD; post-authorization control; reproducibility

1. Introduction

Zero-trust architecture strengthens decisions about access to protected resources, rather than trusting network location alone [1,2]. An allowed request can nevertheless produce an invalid reservation, read an undeclared file, or use a stale service configuration. This paper studies the narrower execution problem: how to combine business-state checks and process resource restrictions while identifying the policy used by each component.

The problem is not that all existing systems stop at authentication. Usage control already includes ongoing decisions, mutable attributes, conditions, and obligations [3]. System-transaction sandboxes can constrain effects spanning multiple system calls [4]. Capability systems and information-flow control address distinct resource and propagation questions [5,6]. PBEA does not supersede those mechanisms. Its proposed contribution is an explicit, implementable contract linking a serial state transition, a worker-lifetime runtime plan, an effective-policy identity, and evidence whose provenance and completeness are stated.

Received:
Revised:
Accepted:
Published:

Citation: Park, S.; Kim, K. Policy-Bound Composition for Stateful Services. *Electronics* **2026**, *1*, 0. https://doi.org/

Copyright: © 2026 by the authors. Submitted to *Electronics* for possible open access publication under the terms and conditions of the Creative Commons Attri- bution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

Two distinctions are central. First, an application invariant and an operating-system capability describe different effect classes. Enabling both controls does not automatically establish that their configurations share a policy identity. Second, equality of recorded identities is not proof that a trusted mapper installed the right operating-system constraints. We separate these properties instead of crediting a digest comparison with remote attestation.

The retained evaluation comprises four configurations and nine deterministic fixtures, not a population of independent real-world attacks. A symmetric ablation shows which control supplies which property. An additional unbound-composition baseline, B3, tests the narrower identity-consistency contract while retaining both protection layers. Its separately frozen implementation produced 540 additional native records and 300 benchmark process blocks; these results are not merged into the original corpus.

The contributions are: (1) a bounded execution contract with separate application and native-failure semantics; (2) a Rust/OpenBSD mapping with worker-policy lifetime checks and explicit trusted components; and (3) reproducible effect-family and boundary evaluation, including honest accounting for missing historical timing samples. We ask whether each plane preserves its intended boundary (RQ1), which identity mismatches a bound composition can detect (RQ2), and what costs and failure boundaries the evidence actually supports (RQ3).

2. Background and Related Work

2.1. Ongoing control, integrity, and contracts

UCONABC is a close conceptual neighbor: it does not restrict authorization to an initial decision [3]. PBEA instead chooses a concrete implementation boundary around candidate-state publication and worker-local resource confinement. It does not implement the full UCON model or show that UCON cannot express such requirements. Design by Contract makes preconditions, postconditions, and invariants explicit [7]; a contract alone does not mediate filesystem or network authority. Runtime monitoring also has principled limits on the policies it can enforce [8]. PBEA's endpoint check remains trusted application code, not an unbypassable kernel destination filter.

Clark–Wilson's constrained data and well-formed transactions are also a close integrity reference [9]. PBEA does not claim to invent application integrity policy. Its runtime plane and policy/evidence contract address a separate composition question, and a small reference implementation does not establish organizational separation-of-duty guarantees. Runtime application self-protection (RASP) is an overlapping engineering area, not a single competing algorithm. OWASP AppSensor provides application-layer detection and automated response [10]; PBEA evaluates a specific candidate-publication and worker-policy contract, without claiming that detection-and-response systems cannot implement comparable guards.

2.2. Transactional sandboxes and capabilities

TxBox uses system transactions to sandbox untrusted application effects [4]. This is stronger than the present implementation with respect to rollback of system resources inside a transaction. PBEA's in-memory candidate is not a system transaction: an allowed network transmission before Abort may remain visible. Capsicum reduces ambient authority through descriptor capabilities [5]; PBEA's pathname restrictions do not revoke all authority in already-open descriptors. Flume governs information flow through OS abstractions [6]; PBEA does not claim general noninterference.

2.3. Current enforcement research

BeeBox hardens the BPF runtime against transient execution attacks and evaluates real BPF workloads [11]. Its threat and enforcement unit differ from PBEA's trusted kernel and serial application state. Such work is not an absent feature to mark as a failure in a comparison matrix. It addresses a different boundary. Supply-chain provenance, such as in-toto [12], likewise does not make a process's domain mutation correct merely by identifying its artifacts.

Table 1. Closest approaches and the distinction investigated here. "Outside scope" is not evidence of inability.

Approach	Primary semantics	Resource/evidence boundary	Relation to PBEA
UCONABC	Ongoing authorization and mutable attributes	Model-level usage control	Broader usage model; not a measured competing implementation
TxBox	System-transaction policy and recovery	Kernel transaction workset	External rollback not supplied by PBEA
Capsicum	Capability-oriented authority	Descriptor rights and capability mode	Resource confinement, not domain-specific invariant semantics
Flume	Information-flow constraints	OS abstractions and labels	Noninterference differs from candidate-state validity
BeeBox	BPF runtime hardening	Kernel BPF execution	Kernel defense outside the present trusted-kernel model
PBEA	Serial domain transition plus policy identity	OpenBSD worker and local supervisor	Evaluated integration contract; no general dominance claim

3. System and Threat Model

The request $q = (a, s, c, u)$ identifies actor, asset, context, and operation. The threat begins after authenticated access is allowed. We trust the Rust verifier, policy source, resolver, runtime mapper, evidence supervisor, cryptographic primitives, kernel, and hypervisor. The adversary controls admitted request contents and, in explicit deployment-drift fixtures, the declared identities or versions supplied to independently managed components. The wrong-install counterexample deliberately violates the trusted-mapper assumption and must not be scored as an attack prevented by an identity comparison.

A worker uses one immutable runtime policy for its lifetime. Several operations may share this policy. Replacing or expanding it requires a new worker. OpenBSD restrictions are process-wide and monotonic; a request cannot restore privileges removed by an earlier pledge, nor extend a locked unveil table [13,14].

Let D_b, D_a be before/after committed domain state, T_b, T_a be trust state, and D^* be a detached candidate. For an application Commit,

$$I(D_b, D^*, u, c) \quad \wedge \quad D_a = \text{install}(D^*). \tag{1}$$

For Reject or Abort, the intended domain property is $D_a = D_b$. Quarantine also leaves the candidate uninstalled but may update T_a according to a separate trust-state rule. Immutable

before-state hashes must be captured before mutation; a reference alias to the live state is insufficient.

Let E^+ denote successful observable runtime effects and $C(P_R)$ the allowed effects of the runtime policy:

$$\text{RuntimeConform}(q, P_R) \equiv E^+(q) \subseteq C(P_R). \tag{2}$$

A denied prohibited attempt does not violate Equation (2). A counterexample to "state integrity implies runtime conformance" is a valid domain transition accompanied by an actually successful undeclared file read in an unconstrained execution, not merely an attempted read. Conversely, permitted resource usage can accompany a domain mutation that violates Equation (1).

3.1. Identity consistency versus enforcement correctness

An effective policy P has a deterministic identifier

$$h_P = \text{SHA256}(\text{PolicyNormalize}(P)). \tag{3}$$

The existing implementation sorts object members and explicitly set-like policy arrays. Ordered arrays remain ordered. This is a policy-specific normalization, not a demonstrated general RFC 8785 implementation. JCS specifies additional JSON number serialization and UTF-16 member-ordering requirements [15]; no general JCS compliance is inferred from the old golden policy digest.

For request, transition, runtime-plan, and evidence identities, define

$$\text{IdentityConsistent} \equiv h_q = h_I = h_R = h_e. \tag{4}$$

An input failing Equation (4) can be correctly rejected. Thus input admissibility and enforcement correctness are different predicates. The operational requirement is that a bound worker reject inconsistent identities before protected dispatch, not that every rejected request satisfy identity consistency. Even four equal identities do not attest the OS configuration: a faulty trusted mapper can install different calls while labeling them identically.

3.2. Application and native outcomes

Application completion has Commit, Reject, Abort, or Quarantine. StartupFailure occurs before readiness. A supervisor may instead observe native termination, with no application outcome. For such a record, the child after-state is unknown unless independently observed; an unchanged parent object is not evidence about the dead child's state. Completion evidence and termination observations therefore have separate origins and completeness flags. None implies durable storage, crash recovery, or tamper-proof distributed logging.

4. PBEA Design

Figure 1 shows startup before admission. Source policy is parsed without duplicate members, schema checked, and resolved for inheritance, defaults, and conflicts. The effective policy generates a digest and a runtime plan. Resource preparation precedes listener readiness, unveil application, unveil lock, and pledge. The protected loop is entered only after installation succeeds. Endpoint allowlists are separately checked in the application before connect.

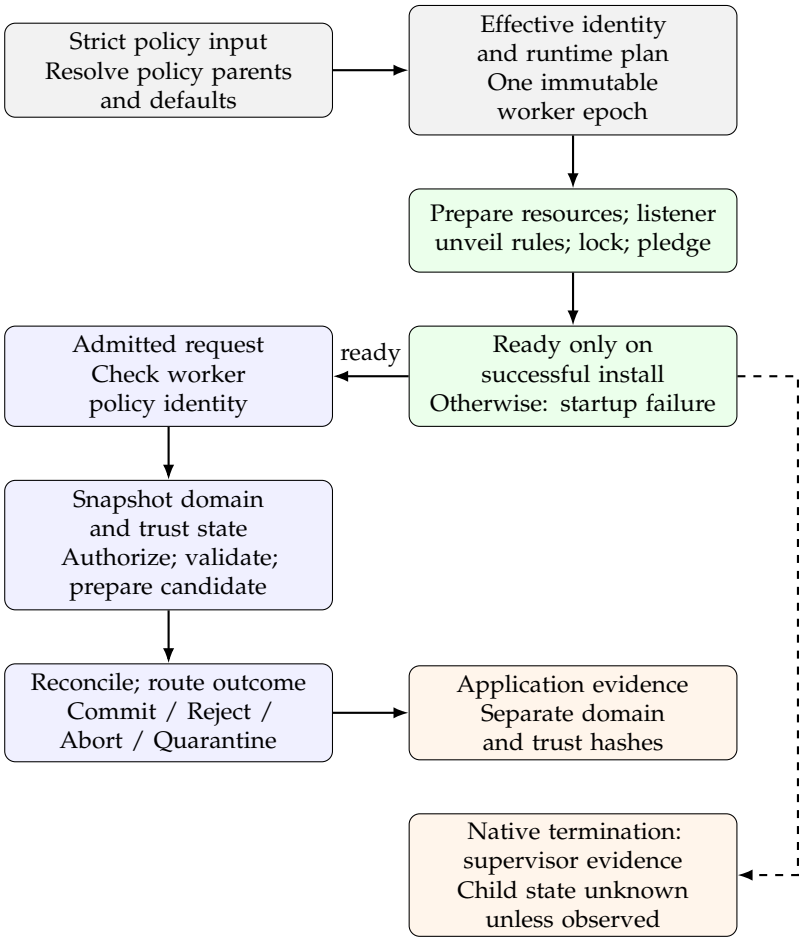


Figure 1. Worker-lifetime policy composition. The supervisor observation is not an application outcome. The same fragment is used by the standalone figure.

The runtime plan records policy identifier, epoch, backend identifier, normalized unveil rules, pledge string, lock requirement, and a plan digest. The added startup guard remembers the prepared policy digest and rejects a different policy at installation. The service emits epoch and plan identifiers on reaching readiness. These are local provenance records, not remote attestation or proof that the mapper is semantically correct.

Algorithm 1 Serial protected completion; native termination is observed externally.

Require: Ready worker, admitted q, committed (D, T), immutable effective policy P
$D_b \leftarrow \text{ImmutableSnapshot}(D)$
$b_D \leftarrow \text{Hash}(D_b)$; $b_T \leftarrow \text{ImmutableHash}(T)$
if request or component identities disagree **then**
    return Evidence(Reject, b_D, Hash(D), b_T, Hash(T))
end if
Authorize and validate the operation and input
if optional trust policy requires quarantine **then**
    Update T according to that policy; do not install a domain candidate
    return Evidence(Quarantine, b_D, Hash(D), b_T, Hash(T))
end if
$D^* \leftarrow \text{PrepareCandidate}(D, q)$
if protected preparation/reconciliation fails **then**
    return Evidence(Abort, b_D, Hash(D), b_T, Hash(T))
end if
if $\neg I(D_b, D^*, u, c)$ **then**
    return Evidence(Reject, b_D, Hash(D), b_T, Hash(T))
end if
$D \leftarrow \text{Install}(D^*)$
return Evidence(Commit, b_D, Hash(D), b_T, Hash(T))

Algorithm 1 specifies the integration contract rather than claiming every optional policy is connected in every example. In the payment example, positive-amount validation occurs before tuple candidate creation and installation is inside reconciliation. The risk-aware core interface additionally separates read-only reconciliation from commit. The revision reservation service uses a detached state candidate and a conservation invariant. These implementation differences do not permit claiming external-I/O transactionality.

5. Implementation

The workspace retains the historical `poa-*` crate prefixes. The core owns typed transition outcomes; the protocol crate owns strict policy types, resolution, and normalization; the sandbox crate owns platform calls; the example crate owns domain hooks and orchestration. The revision does not rewrite the domain-blind core.

Table 2. Concrete enforcement mapping and trust boundaries.

Policy element	Concrete implementation	Limit or failure rule
Unveil path and `r/w/x/c`	Read, write, execute, create/remove pathname rights	Directories apply to descendants; no general revocation of existing FDs
Empty unveil set	Mask root, then lock	Lock alone does not establish deny-all
Lock requirement	`unveil(NULL,NULL)` after rules	Failure prevents readiness
Pledge promises	Validated, sorted promise string	Process-wide syscall classes, not destinations
Network endpoints	Application IP/port/protocol predicate	Trusted check; raw socket code could bypass it
Policy epoch	One effective policy per worker	New policy requires new worker
Prepared policy	Digest compared before install	Detects declared skew, not dishonest installation metadata
Evidence	Application or supervisor writer	Child state can be unobserved

Strict parsing now rejects duplicate members before deserialization loses them. Tests cover nested duplicates, trailing input, invalid numeric/Unicode encodings, ordered-array preservation, set-like-array order invariance, and preserving distinct Unicode sequences. Existing schema, inheritance conflict, privilege-expansion, and golden digest regressions are retained. This is a restricted implementation profile; unsupported JSON semantics must not be silently called universal canonicalization.

The service opens its audit descriptor and listener before confinement. This is intentional but security-relevant: pathname visibility is not an exhaustive descriptor capability inventory. The additional FD fixture opens a synthetic outside file before installing restrictions, then compares descriptor use with a new pathname open. The current worker does not promise a complete inherited-descriptor sanitizer. Allowed child execution after exec is not evaluated; the exec-denial result does not prove safe post-exec privilege inheritance.

The reservation service starts with ten units, tracks availability and a reservation map, and enforces $available + \sum reserved = total$. It rejects duplicate reservation IDs, zero quantities, insufficient stock, and cancellation of unknown IDs. Its sequence includes normal reservation, oversubscription, duplicate input, candidate Abort, cancellation, and repeated cancellation. It is serial and in memory. No serializability theorem, persistent quarantine, OS-crash rollback, or distributed transaction is claimed.

6. Evaluation Methodology

6.1. Preserved historical corpus

The frozen evaluation source is 62229cd31ad1915cdd850fe3cfd670cddf208d0c; the reporting correction is f2be0bfbcbf2aad093018d69e513cc61b169e5e3. The original three manuscript inputs are hash-preserved. Four raw JSONL files contain $4 \times 9 \times 30 = 1080$ records. The reanalysis reproduces the canonical metrics and verifies all 37 files of the retained manifest after mapping relocated paths.

B0 performs direct mutation after admitted access; B1 adds transition and digest checks; B2 installs runtime confinement without the transition guard; P combines them. B0 is not representative of every ZTA or UCON implementation. S0 is benign; S1 violates a positive-amount invariant; S2–S4 test file read, helper execution, and a controlled prohibited endpoint; S5 alternates malformed policy and missing resource; S6 repeats a wrong-digest

request; S7 injects a candidate-stage fault where that stage exists; S8 changes the request digest.

S7 is N/A in B0/B2: the direct-mutation path does not execute the after-candidate fault. Its normal state change is neither a rollback failure nor a malicious effect. B1/P abort without installing the candidate. S5 is reported as separate 15-run causes rather than an undifferentiated 30-case probability.

The archived S3 supervisor spawns an isolated child that installs restrictions and attempts helper execution. The supervisor records signal and marker presence. Its local parent-domain hashes must not be interpreted as child-state after-images. An additive observation view retains original record IDs while marking the application outcome and child after-state unobserved for native termination.

6.2. Unbound composition and additional boundaries

B3 retains transition checks and the same runtime backend but manages component identities independently. P applies the equality contract before dispatch. Prespecified cases include aligned identities, runtime epoch skew, stale evidence identity, resolved-policy drift, request/worker epoch mismatch, and equal IDs with intentionally wrong installation. The last case tests the limitation of metadata consistency, not a property the design is expected to enforce.

Additional cases test an allowed receiver-visible transmission followed by candidate Abort, authority retained by a preopened descriptor, and the reservation sequence. The supervisor retains child failures instead of dropping them. The new validator checks completeness, uniqueness, provenance, and field consistency; an observed security failure is valid data, not grounds to reject a record. The prespecified 9 cases $\times$ 2 modes $\times$ 30 repeats produced 540 observed records. This modest integration comparison does not make B3 representative of all independently managed security systems.

6.3. Performance and experimental units

The historical S0 benchmark ran 100 warm-up and 5000 measured operations per mode in a long-lived process. It retained aggregate latency statistics, not individual timings. Its functional-equivalence artifact compares operation, normal result, and a normalized success label; it is not a wire-byte equivalence test. The original direct path performs a map insertion, whereas guarded paths additionally parse and validate the request and generate security evidence. Therefore their cost difference includes security-specific work.

The revision runner stores raw nanosecond samples, empty-timer measurements, a rotating mode order, and 20 independent process blocks per mode/workload. Each block warms up 100 operations and measures 500. Workloads include one asset, 128 assets, and a permitted file read. Sequential samples within a block are not independent experimental units. Quantiles and block-median ranges are reported separately. Neither throughput nor memory scaling is inferred without measurement; the payment candidate remains a tuple rather than a whole-state copy.

7. Results

7.1. Reproduced effect-family results

Table 3 is generated from the revalidated historical corpus. It supports complementary protection boundaries, not a population-level attack success estimate. Transition-only prevents the tested S1 effect but not S2–S4; Runtime-only does the reverse. Full-PBEA combines them in the evaluated fixture. S5's split shows why parsing and runtime-resource readiness are not interchangeable.

Table 3. Historical results, as numerator/denominator. N/A excludes an absent fault point.

Metric	B0	B1	B2	P
S1 invalid state	30/30	0/30	30/30	0/30
S7 rollback failure	N/A	0/30	N/A	0/30
S2–S4 containment	0/90	0/90	90/90	90/90
S8 mismatch accepted	30/30	0/30	30/30	0/30
S5 malformed failure	0/15	15/15	15/15	15/15
S5 missing-resource failure	0/15	0/15	15/15	15/15
S6 quarantine	0/30	30/30	0/30	30/30
S0 benign success	30/30	30/30	30/30	30/30

Across the 90 P S2–S4 attempts, no prohibited effect succeeded. Filesystem denial, helper termination, and application endpoint rejection are distinct observations. The endpoint result is not evidence that pledge filters a destination IP. BT1 permits delivery to the explicitly allowed endpoint. BT2 denies the tested symlink traversal in B2/P, but does not establish a general race-free filesystem theorem. Repetitions demonstrate stability of these deterministic tests, not 1080 unique attack inputs.

7.2. Historical cost and unavailable uncertainty

Table 4 regenerates the archived steady-state summaries and derives both absolute increment and ratio. Full-PBEA's median is below one millisecond yet approximately 264.28 times the small direct-insertion baseline. Both observations matter. P being faster than B1 in this one schedule does not show that confinement improves speed.

Table 4. Historical synthetic S0 summaries in microseconds; new independent-block measurements are a separate corpus.

Mode	P50 (μs)	P95	P99	Δ B0	Ratio B0
B0	0.700	0.701	1.000	0.000	1.00
B1	208.195	262.393	378.591	207.495	297.42
B2	0.700	0.701	0.800	0.000	1.00
P	184.995	218.495	320.293	184.295	264.28

B0 and B2 have identical reported P50/P95 values (0.700/0.701 microseconds) after restrictions were installed. This shows no measurable difference on the synthetic, nearly syscall-free path at the reported resolution; it says nothing about zero cost for real I/O. The missing historical sample series prevents reconstructing block uncertainty or independently checking the original quantiles. The archived startup corpus contains 30 observations per mode, with medians approximately 44 ms. No startup ranking is claimed.

7.3. Additive native results

Table 5. Additional B3/P observations from the separately frozen revision source.

Configuration	Runtime-ID skew accepted	Equal-ID mapper defect effect
B3	30/30	30/30
P	0/30	30/30

For aligned policies both modes committed 30/30 requests. Across the four declared mismatch cases, B3 committed all 120 and P rejected all 120 before dispatch. This supports detection of supplied identity skew, not discovery of arbitrary semantic policy drift. In the intentionally wrong-install case, both modes completed all 30 forbidden outside-file reads

despite equal claimed IDs. This observed counterexample is retained, narrowing the claim to identity consistency under trusted mapping.

For allowed-send-before-Abort, the receiving socket observed all 30 transmissions per mode while the candidate domain state remained uninstalled. For the preopened descriptor fixture, both modes read the file through the existing handle in all 30 trials, while a new pathname read was denied. The reservation sequence preserved its conservation invariant and expected operation sequence in all 30 trials per mode. These outcomes distinguish transaction publication, external effects, and descriptor authority.

The new performance corpus contains 20 independent processes for each of five modes and three workloads. Table 6 reports pooled descriptive quantiles, not independent-sample uncertainty. For the one-asset path, P's process-block medians ranged from 186.595 to 232.644 microseconds. The corresponding ranges overlap those of B1/B3; no speedup caused by confinement is inferred. The full block statistics and empty-timer observations remain in the canonical JSON artifact.

Table 6. New native timing summaries in microseconds, generated from raw samples. Each row represents 20 independent process blocks and 10,000 measured sequential operations.

Workload	Mode	P50	P95	P99	Δ B0	Ratio B0
state1	B0	0.600	0.700	0.899	0.000	1.00
state1	B1	222.895	304.792	430.989	222.295	371.49
state1	B2	0.600	0.700	0.899	0.000	1.00
state1	P	217.494	298.993	407.191	216.894	362.49
state1	B3	214.895	290.993	399.690	214.295	358.16
state128	B0	0.700	0.800	2.700	0.000	1.00
state128	B1	216.294	289.093	396.591	215.594	308.99
state128	B2	0.700	0.701	0.900	0.000	1.00
state128	P	217.595	290.792	404.790	216.895	310.85
state128	B3	215.495	290.293	415.690	214.795	307.85
allowed-io	B0	4.200	4.700	8.000	0.000	1.00
allowed-io	B1	231.294	316.292	433.190	227.094	55.07
allowed-io	B2	4.200	5.300	9.100	0.000	1.00
allowed-io	P	227.895	305.893	425.589	223.695	54.26
allowed-io	B3	223.795	305.292	424.889	219.595	53.28

8. Discussion and Limitations

8.1. What policy binding adds

The implemented contract can detect declared disagreement between component identities and a policy change between resource preparation and installation. It cannot independently attest that trusted code used the intended system calls. B3 testing is therefore a test of deployment identity consistency under specified skew, not proof of a universal new principle. If independently managed layers carry consistent identities, B3 and P should have the same benign outcome. The architecture's novelty is bounded integration and traceability, not the discovery that two security layers can be composed.

8.2. State, external effects, and failures

Rejecting a candidate is not rollback of the entire service environment. An allowed send can reach a receiver before a later failure, and a descriptor opened before confinement may retain authority. External-effect buffering, outboxes, compensation, persistent state, and restart recovery would need separate implementation and evaluation. The relevant failure models are ordinary function error, child termination, service death, and OS crash; only the measured model can be credited.

8.3. Threats to validity

The historical experiment uses one OpenBSD 7.9 VM and a small payment fixture. Workload representativeness, privilege-authoring quality, hidden library authority, inherited descriptors, mapper correctness, and audit durability remain limits. B0/B1/B2 are chosen configurations, not commercial systems. No Linux backend claim follows from a compiling placeholder. RiskEvidence is optional, and the payment example's repeated-violation counter is not a persistent distributed quarantine service. A trusted supervisor can misreport; its evidence is not a cryptographic attestation chain.

The revised tests expand distinct causes and business sequencing, not merely repetitions. Nonetheless they remain controlled mechanism tests. A 30/30 result is not a probability bound over real attacks, even if a binomial interval can be mechanically calculated. No claim about prompt injection, arbitrary malware, malicious administrators, or compromised kernels is supported.

Table 7. Claim–evidence boundaries.

Claim	Evidence location	Restriction
Protected domain publication	S1, applicable S7; reservation unit test	Serial memory, no external-I/O rollback
Prohibited capabilities contained	Historical S2–S4 and native errno/signal	App endpoint check is TCB
Policy identity consistency	Worker startup guard and binding tests	No attestation of actual kernel plan
Normal behavior preserved	S0 hashes and normalized success	Not identical wire response payloads
Quantified implementation cost	Archived summaries; separate new raw blocks	Historical sample-level uncertainty unavailable

9. Conclusions

PBEA specifies how a serial stateful service can combine candidate-state integrity, process resource confinement, policy identity, and explicit evidence semantics. The retained four-mode corpus supports distinct, non-substitutable enforcement roles in its fixtures. The revision narrows identity claims, distinguishes native termination from application completion, and makes the worker lifetime explicit. It also exposes the limits of domain-state rollback and historical performance data. The intended result is an auditable implementation contract and reproducible bounded evaluation, not universal prevention or superiority over existing usage control, capability, or transactional systems.

Author Contributions: Author-specific CRediT roles and final approval must be confirmed by the authors before submission.

Funding: The original draft reports IITP/ITRC support, grant IITP-2026-RS-2021-II211835. The authors must confirm the funding statement for the submission version.

Institutional Review Board Statement: Not applicable to the synthetic software fixtures described here; author confirmation is required for final submission.

Informed Consent Statement: Not applicable to the synthetic software fixtures described here.

Data Availability Statement: Historical raw evidence and newly generated local revision artifacts are retained in the project evidence package. A public archival release of this revision has not been verified. The authors must provide a reviewer-accessible release before submission. This study created and analyzed data; data sharing is not inapplicable.

Acknowledgments: Generative AI agents assisted code, analysis, and manuscript revision. Exact tool/version disclosures and author responsibility statements require author review before submission.

Conflicts of Interest: The authors must confirm their conflicts-of-interest declaration before submission.

References

1. Rose, S.; Borchert, O.; Mitchell, S.; Connelly, S. Zero Trust Architecture. NIST Special Publication 800-207, National Institute of Standards and Technology, 2020. https://doi.org/10.6028/NIST.SP.800-207.
2. Chandramouli, R.; Butcher, Z. A Zero Trust Architecture Model for Access Control in Cloud-Native Applications in Multi-Location Environments. NIST Special Publication 800-207A, National Institute of Standards and Technology, 2023. https://doi.org/10.6028/NIST.SP.800-207A.
3. Park, J.; Sandhu, R. The UCONABC Usage Control Model. *ACM Transactions on Information and System Security* **2004**, *7*, 128–174. https://doi.org/10.1145/984334.984339.
4. Jana, S.; Porter, D.E.; Shmatikov, V. TxBox: Building Secure, Efficient Sandboxes with System Transactions. In Proceedings of the 2011 IEEE Symposium on Security and Privacy, 2011, pp. 329–344. https://doi.org/10.1109/SP.2011.33.
5. Watson, R.N.M.; Anderson, J.; Laurie, B.; Kennaway, K. Capsicum: Practical Capabilities for UNIX. In Proceedings of the 19th USENIX Security Symposium (USENIX Security 10), Washington, DC, August 2010.
6. Krohn, M.; Yip, A.; Brodsky, M.; Cliffer, N.; Kaashoek, M.F.; Kohler, E.; Morris, R. Information Flow Control for Standard OS Abstractions. In Proceedings of the Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP), 2007, pp. 321–334. https://doi.org/10.1145/1294261.1294293.
7. Meyer, B. Applying "Design by Contract". *Computer* **1992**, *25*, 40–51. https://doi.org/10.1109/2.161279.
8. Schneider, F.B. Enforceable Security Policies. *ACM Transactions on Information and System Security* **2000**, *3*, 30–50. https://doi.org/10.1145/353323.353382.
9. Clark, D.D.; Wilson, D.R. A Comparison of Commercial and Military Computer Security Policies. In Proceedings of the 1987 IEEE Symposium on Security and Privacy, 1987, pp. 184–195. https://doi.org/10.1109/SP.1987.10001.
10. OWASP Foundation. OWASP AppSensor. Accessed 20 October 2026.
11. Jin, D.; Gaidis, A.J.; Kemerlis, V.P. BeeBox: Hardening BPF against Transient Execution Attacks. In Proceedings of the 33rd USENIX Security Symposium. USENIX Association, 2024, pp. 613–630.
12. Torres-Arias, S.; Afzali, H.; Kuppusamy, T.K.; Curtmola, R.; Cappos, J. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In Proceedings of the 28th USENIX Security Symposium (USENIX Security 19), Santa Clara, CA, August 2019; pp. 1393–1410.
13. OpenBSD Project. `pledge(2)`—Restrict System Operations. OpenBSD Manual Pages, 2026. Evaluated platform: OpenBSD 7.9.
14. OpenBSD Project. `unveil(2)`—Unveil Parts of a Restricted Filesystem View. OpenBSD Manual Pages, 2026. Evaluated platform: OpenBSD 7.9.
15. Rundgren, A.; Jordan, B.; Erdtman, S. JSON Canonicalization Scheme (JCS). RFC 8785, RFC Editor, 2020. https://doi.org/10.17487/RFC8785.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.